

Ordo / HeyOrDo Interview Preparation Guide

Pratik Burkhawala | Master preparation roadmap

Introduction → Company → Resume Deep Dive → Technical → Product Design → Behavioral → Questions

Purpose

This guide is tailored to the shortlisted Ordo/HeyOrDo role and to the experience presented in the current resume. It combines preparation for the first introduction call with deeper preparation for later technical and product rounds. It deliberately separates confirmed experience from areas where you should be honest about limited hands-on exposure.

1. Interview Strategy at a Glance

Area	Priority	Your position
Introduction / motivation	Very high	Prepare a crisp 1.5–2 minute story
Ordo company/product	Very high	Understand product, architecture, and why your experience transfers
Android architecture	Very high	Strong
BLE / GATT	Very high	Strong
OTA / firmware delivery	Very high	Extremely strong
UDS / ECU communication	Very high	Extremely strong
Android system internals	Very high	Strong
Performance / debugging	Very high	Strong
Kotlin / Coroutines / Flow	High	Strong
Backend / real-time communication	High	Good
Release / telemetry / reliability	High	Good
Audio / AI / privacy	Medium	Partial; prepare gap answers
Coding	Medium for intro; higher later	Light refresh now
Product system design	High	Prepare from scratch exercise
Behavioral / ownership	High	Prepare 5 stories

2. Opening: Your Introduction

Target: 1.5–2 minutes. Do not walk through every job chronologically. Tell a progression story: Android → embedded/connectivity → Matter → end-to-end ownership → why Ordo.

Q: Tell me about yourself.

A: I'm a Lead Engineer with 8+ years of experience building software for Android, embedded, and connected-device platforms. I'm strong in system-level Android, Bluetooth/BLE, UDS, and OTA, with hands-on ownership across architecture, implementation, device integration, and production delivery. In my current role at Matter Motor Works, I own critical capabilities across the vehicle infotainment platform, including the end-to-end OTA flow from cloud to Android to ECU, along with connectivity and system-level Android work. I'm now looking for a role where I can apply that connected-device experience to a new product category that combines Android, hardware, AI, and a new user interface.

3. Motivation / Non-technical Questions

Q: *Why are you looking for a change?*

A: I'm looking for greater technical and product ownership, with the opportunity to work on complex Android, connected-device and AI-driven products and contribute from architecture through production.

Q: *Why Ordo?*

A: What attracted me is the combination of Android, connected hardware, BLE, audio, AI and a completely new user interface. My recent work has been at the intersection of Android and physical devices, particularly OTA, BLE, system-level Android and cloud-to-device architecture. I see this as a natural extension of that experience into an AI-native product.

Q: *Why move from automotive to AI wearables?*

A: The domain is different, but the underlying engineering problems are very similar: Android acting as the bridge between software and hardware, reliable wireless connectivity, device lifecycle, firmware, cloud communication and performance. I'm interested in applying that experience to a new kind of consumer interface.

Q: *What are you looking for in your next role?*

A: A product-focused environment where I can own a meaningful technical surface area, make architecture decisions, work closely with hardware and software teams, and take features from concept through production.

4. Know Ordo / HeyOrDo

Current public positioning: Ordo describes itself as an ear-worn AI interface designed to let AI see, hear, speak and remember without putting another screen between the user and the world. Public product material highlights a built-in camera, voice interaction, memory, visual intelligence, open-ear design, and app integrations. The public site says visual intelligence uses an on-device AI model on the smartphone. LinkedIn currently lists Ordo as a 2–10 person computer-hardware company founded in 2026.

- Core product idea: remove the screen from AI interaction and make the ear-worn device the interface.
- Built-in camera: enables hands-free visual capture and visual understanding.
- Voice interaction: hear the user and respond through audio.
- Memory: retain information and make it retrievable later.
- Visual intelligence: public site says an on-device model on the smartphone is used.
- App integrations: public materials mention services such as Slack, Notion and Gmail.
- Open-ear design: aims to keep the user aware of surroundings.
- Early-stage environment: expect broad ownership, ambiguity, rapid iteration, and direct product impact.

Why your background fits

Do not think of this as “automotive Android engineer moving to earbuds.” Think of it as “Android + connected-device engineer applying an existing hardware/software bridge to a new AI interface.”

- Matter: Cloud → Android → ECU. Ordo: Cloud/AI → Android → wearable device.
- Both require reliable wireless connectivity, device state, firmware lifecycle, failure recovery, production telemetry, and Android as the control plane.
- Your strongest transferable areas are BLE/GATT, OTA, UDS, Android system work, cloud/device architecture, performance, and field debugging.

5. Resume Deep Dive: Matter Motor Works

Be ready to explain each of these without referring to notes.

OTA architecture

- Architected the end-to-end Cloud → Android → ECU OTA platform.
- Defined firmware package creation, cloud storage, approval/staging workflow, versioning, delivery, verification, recovery and rollback.
- Developed the Android OTA implementation.
- Handled download, verification, resumability, retries, progress, version validation, failure recovery and rollback.
- Achieved approximately 90% OTA success rate.

UDS / ECU

- Implemented the UDS client for ECU communication and flashing.
- Prepare to explain Diagnostic Session Control, ECU Reset, Security Access, Read/Write Data by Identifier, Routine Control, Request Download, Transfer Data, Transfer Exit, Tester Present, positive/negative responses, timeout/retry handling, and recovery.

Android architecture

- Kotlin, Coroutines, Flow / StateFlow / SharedFlow.
- MVVM, Clean Architecture, dependency injection, repository/use-case patterns, modularization.
- Jetpack Compose: exposure/experience, but not your deepest production area.

Android system / platform

- Binder IPC, AIDL, Android system services, system/privileged apps, platform APIs, custom Android builds, device/vendor integration, boot/startup, permissions, background services.
- Accessibility: earlier TalkBack-focused application accessibility experience; do not claim direct AccessibilityService implementation unless confirmed.

Connectivity

- BLE scanning, pairing, bonding, GATT client/server, services, characteristics, notifications/indications, reconnection, connection state management, custom protocol.
- Wi-Fi, CAN, Bluetooth audio / A2DP.

Backend / cloud

- REST APIs, MQTT, Azure Functions, MongoDB, PostgreSQL.
- Databricks exposure/usage, not a core implementation claim.

Performance / production

- Perfetto-based root-cause analysis leading to approximately 20% performance improvement.
- Crash/telemetry, vendor coordination, production deployment, field debugging.

Leadership

- Owns capabilities end-to-end and mentors 1–2 engineers directly while guiding engineers across teams.

6. Android Architecture Preparation

Know these concepts

- MVVM and why it helps separate UI state and business logic.
- Clean Architecture: presentation, domain, data boundaries.
- Repository and Use Case responsibilities.
- Dependency inversion and dependency injection.
- Modularization: feature/core/domain/data modules; build boundaries; ownership.
- Testability: pure business logic, mockable repositories, unit/integration/UI testing.

Kotlin / Coroutines / Flow

- suspend functions, structured concurrency, cancellation, exception handling, Dispatchers, coroutine scopes.
- Flow vs StateFlow vs SharedFlow; cold vs hot streams; collect vs collectLatest; lifecycle-aware collection.

Q: Why Flow instead of callbacks?

A: It gives a structured way to represent ongoing asynchronous state or events, works naturally with coroutines, supports operators and cancellation, and fits reactive UI/state updates. The specific choice depends on whether I need state semantics or one-off/shared events.

Q: How would you structure an OTA feature using Clean Architecture?

A: The UI/ViewModel observes state, use cases own business operations, repositories abstract firmware metadata/download/device operations, and the lower layers handle networking, storage, verification and UDS/device communication. Long-running state should be persisted so interruption or process death can be recovered.

7. BLE / GATT Preparation

Know the theory and connect it to your real experience.

- GAP vs GATT; central vs peripheral; advertising vs scanning.
- Pairing vs bonding; BLE security basics.
- Services, characteristics, descriptors, read/write, notification, indication, CCCD.
- MTU, packet size, connection interval, PHY, RSSI.
- GATT operation sequencing and the practical need to avoid conflicting concurrent operations.
- Reconnect state machine, pending operations, timeout, retry, duplicate packet handling.
- Custom protocol design: command, sequence number, payload, checksum/integrity, ACK/NACK, timeout, retry, state transition.

Q: Notification vs indication?

A: A notification is sent without an acknowledgement at the ATT layer; an indication requires confirmation. The trade-off is reliability versus throughput/latency.

Q: How would you design reconnect handling?

A: Persist the intended device state, model connection states explicitly, avoid issuing operations until the GATT state is valid, restore subscriptions after reconnection, retry only idempotent/safe operations, and expose failure clearly to the upper layer.

8. OTA / Firmware Delivery Deep Dive

Memorize the architecture flow conceptually:

Firmware build/package → cloud storage → approval/staging → eligibility/version check → Android download → verification → UDS/ECU transfer → validation → success/recovery/rollback.

Questions to prepare

- Why did you put Android in the OTA path?
- What happens if the phone loses network connectivity?
- What happens if the ECU fails halfway through flashing?
- How does resume work?
- How do you prevent installation of incompatible firmware?
- What does rollback mean in your architecture?
- How did you stage approvals?
- How did you measure the 90% success rate?

A crisp OTA answer

A: I designed the Cloud → Android → ECU flow, including firmware lifecycle and approval/staging on the cloud side, Android download and validation, and ECU programming through a UDS client that I implemented. I also handled reliability mechanisms such as retries, resumability, progress, failure recovery and rollback, and took the system through production deployment and field debugging.

9. UDS / ECU Preparation

Area	Know it conceptually
Session	Default/programming/extended session concepts; Diagnostic Session Control
Security	Security Access purpose and challenge/response concept
Transfer	Request Download → Transfer Data → Transfer Exit
Control	Routine Control; ECU Reset; Tester Present
Data	Read/Write Data by Identifier
Responses	Positive response; negative response; NRCs
Reliability	Timeouts, retries, session state, ECU recovery
Transport	CAN and where ISO-TP fits in the communication stack

Q: Why UDS for firmware update?

A: UDS provides standardized diagnostic services and a defined diagnostic/transfer flow for communicating with ECUs. In the OTA system I implemented the Android-side UDS client and used it to drive the ECU programming process.

10. Android System Internals

- Binder: Android IPC mechanism, process boundaries, transactions, threading considerations.
- AIDL: define IPC interfaces between client and service processes.
- System services: privileged/platform functionality exposed through system-level components.
- System/privileged apps and platform APIs.
- Background execution: foreground services, WorkManager, Doze, App Standby, process death.

Q: What is Binder?

A: Binder is Android's primary IPC mechanism. It allows processes to call across process boundaries through Binder transactions, with Android managing the underlying IPC and thread handling. The important practical issues are transaction cost, thread behavior, lifecycle, and avoiding blocking work in the wrong context.

11. Performance / Debugging

Your 20% improvement should be a detailed story.

Structure: Problem → Measurement → Perfetto trace → Root cause → Change → Validation → Result.

- Be able to name the exact service/problem area.
- Explain what the trace showed.
- Explain the change you made.
- Explain how you measured the ~20% improvement.
- Refresh CPU, memory, GC, thread contention, Binder latency, ANR and startup concepts.

12. Backend / Real-time / Cloud

- REST: methods, status codes, authentication, retries, idempotency, versioning.
- MQTT: broker, topics, QoS, retained messages, reconnect, subscriptions.
- WebSocket: persistent bidirectional connection, reconnect, heartbeat, ordering.
- Streaming APIs can include WebSockets, SSE and gRPC streaming; they continuously deliver data rather than a single request/response.

Q: How would you honestly describe your streaming experience?

A: My strongest recent production experience is REST and MQTT. I also have earlier WebSocket experience from a chat application. I understand the architecture of real-time communication, but streaming APIs are not the area where I have had the deepest recent production ownership.

13. Release / Telemetry / Reliability

- Crash reporting and ANR investigation.
- Telemetry: collect enough to debug and measure without over-collecting sensitive data.
- Staged rollout, feature flags, remote configuration, rollback.
- OTA success/failure metrics, retry rate, device compatibility, crash-free sessions.

Q: How would you safely roll out a risky feature?

A: Use a gated rollout, monitor crash/performance/feature metrics, keep a remote kill switch or configuration path where appropriate, expand gradually, and have a rollback or disable path ready before broad release.

14. Privacy / Permissions / Accessibility

Know

- Runtime permissions; Bluetooth/camera/microphone permissions.
- Least privilege and data minimization.
- Explicit consent and clear user-visible states.
- Local vs cloud processing and privacy trade-offs.
- Encryption and deletion flows.
- TalkBack basics: content descriptions, accessibility focus, traversal, custom view accessibility and testing.

Q: What is your accessibility experience?

A: Earlier in my Android experience I worked on making an application accessible for users relying on TalkBack, including improving accessibility behavior of the UI. I would describe that as practical Android accessibility experience; I have not recently owned a dedicated AccessibilityService.

15. Ordo Product: Design the Android App from Scratch

This is the most important product-system-design exercise to prepare.

Start with requirements

- Device onboarding, pairing, connection state, battery, configuration, firmware.
- Camera capture and visual intelligence.
- Voice capture and audio playback.
- AI interaction and response.
- Memory: capture, store, retrieve, delete.
- App integrations such as Slack, Gmail, Notion.
- Permissions, consent, privacy, encryption and deletion.
- Telemetry, reliability, crash handling and release management.

Proposed module view

app
device (BLE / pairing / protocol / firmware)
audio
camera
ai
memory
integrations
account
privacy
telemetry
core (networking / database / logging / security)

Architecture pattern

UI → ViewModel → Use Case → Repository → Data/Device/API layer

- Keep BLE protocol details below a device abstraction.
- Keep AI provider details behind an AI interface so the app can evolve.
- Persist long-running operation state so process death does not corrupt device state.
- Separate UI, domain logic, data, hardware communication and telemetry.

BLE device architecture

DeviceManager → ConnectionManager → GATT Client → Protocol Layer → Command Queue → Device

- Explicit connection state machine.
- Serialize GATT operations as required by the platform/device protocol.
- Restore subscriptions after reconnect.
- Retry only safe/idempotent operations.
- Version custom protocol messages.

Voice path

User speaks → microphone → wake word → audio capture → Android/cloud AI → response → audio playback → earbud

Be prepared to discuss where processing happens, latency, audio focus, buffering, privacy and offline degradation.

Visual intelligence path

Camera → capture → transfer → pre-processing → on-device model → inference → result → AI response → audio

- Trade-offs: latency, bandwidth, memory, CPU/GPU/NPU, thermal behavior, privacy.

Memory path

Voice/event → capture → transcription/processing → memory object → encrypted storage → index/retrieval → AI context

- Ask: what is stored, where, how it is encrypted, how it is deleted, how it syncs, how offline behavior works.

Reliability scenarios

- BLE disconnects → reconnect, restore state, retry safe operations.
- Internet disappears → queue/retry/degrade gracefully.
- AI unavailable → fallback and clear user feedback.
- Firmware update interrupted → resume/recovery.
- App process killed → persisted state and safe recovery.
- Battery low → reduce workload / graceful degradation.
- Device overheats → reduce duty cycle/workload.

16. Product Trade-offs

Decision	Key trade-off
BLE vs Wi-Fi	Power and range vs bandwidth
On-device AI vs cloud AI	Privacy/latency/offline vs model size/compute
Battery vs functionality	Camera + audio + AI can be power intensive
Reliability vs latency	Retries and recovery can increase latency
Modularity vs speed	More boundaries improve maintainability but add initial complexity

17. Gaps in the JD: How to Answer

Use the pattern: acknowledge what you have not done directly → state the closest real experience → demonstrate that you understand the concept → show how you would ramp up.

JD gap	Your honest position	Answer strategy
Jetpack Compose	Some experience, not deepest production area	Comfortable with fundamentals; strongest production experience is broader Android/system work.
Advanced audio DSP	A2DP/Bluetooth audio; no HFP/echo/noise/wake-word ownership	State the boundary clearly; show understanding of the audio path.
TFLite/LiteRT / ONNX / QNN	No direct production experience	Mention ML Kit on-device ML; understand model loading/inference/memory/acceleration at a conceptual level.
Streaming APIs	Earlier WebSocket exposure; REST/MQTT stronger	Be clear that streaming is not your deepest recent production area.
AccessibilityService	TalkBack/application accessibility; not confirmed direct service implementation	Describe TalkBack work and avoid claiming dedicated AccessibilityService implementation.

MediaProjection	No direct experience established	Say so; explain adjacent Android system/background experience.
Echo cancellation / noise suppression / wake word	No direct experience	Explain that audio DSP is a growth area and your stronger area is connectivity/device integration.
Thermal-aware ML scheduling	No direct experience	Discuss general performance/thermal trade-offs without claiming production ownership.

Q: Have you worked with TFLite?

A: I have integrated on-device ML through Google ML Kit, including barcode scanning, but I have not directly worked with TFLite/LiteRT, ONNX Runtime or Qualcomm QNN in production. I understand the broader considerations around model loading, inference latency, memory and hardware acceleration, and I would like to go deeper into that area.

Q: Have you worked on audio DSP?

A: I have experience with Bluetooth audio and A2DP integration, but my production experience does not extend deeply into DSP areas such as echo cancellation, noise suppression or wake-word engines.

18. Behavioral / Ownership Stories

Prepare five real stories using Situation → Task → Action → Result.

- OTA: complete architecture and delivery ownership.
- Performance: Perfetto → root cause → fix → ~20% improvement.
- BLE/connectivity: difficult connection/protocol problem and recovery.
- Vendor/cross-team challenge: unclear ownership, dependency, or platform issue.
- Leadership: mentoring engineers and guiding work across teams.

Likely behavioral questions

- Tell me about something you owned end-to-end.
- Tell me about a difficult production issue.
- Tell me about a failure and what you changed afterward.
- Tell me about a technical disagreement.
- How do you prioritize when requirements are unclear?
- How do you mentor other engineers?
- Tell me about a decision you made with incomplete information.
- What would your teammates say you are strongest at?

19. Coding Preparation

For an introduction call, keep coding preparation light. If a dedicated coding round is scheduled later, increase the depth.

- Kotlin syntax and idioms.
- Arrays, strings, HashMap/HashSet, stacks/queues.
- Two-pointer and simple sliding-window problems.
- Sorting and basic Big-O.
- Simple concurrency and coroutine questions.

- 10–15 easy Kotlin problems is enough for the immediate screen; focus more on your own technical stories right now.

20. Questions You Should Ask Them

- “What would you expect this person to own in the first 3–6 months?”
- “How is the Android team structured today, and how does Android interact with the hardware, firmware and AI teams?”
- “What are the biggest technical challenges you are currently facing with the companion app or device platform?”

21. Final Mental Model

Do not frame yourself as “an automotive Android engineer trying to move into AI earbuds.” Frame yourself as an Android + connected-device engineer who has already built software sitting between cloud systems and physical hardware, now applying that capability to a new AI-native human-computer interface.

22. Final Preparation Order

Order	Prepare
1	Introduction, why change, why Ordo
2	Company/product understanding
3	OTA and UDS deep dive
4	BLE/GATT deep dive
5	Android architecture + Kotlin/Coroutines/Flow
6	Android internals: Binder/AIDL/system services
7	Perfetto/performance + telemetry
8	Backend/REST/MQTT/WebSocket
9	Privacy/permissions/accessibility/audio/AI gaps
10	Ordo Android app system design from scratch
11	Behavioral stories
12	Light coding refresh

23. Company Research Notes / Sources

These notes are based on the current public Ordo/HeyOrDo website and LinkedIn material reviewed for interview preparation. The public site may change, and Ordo states that the product is still based on engineering prototypes and final specifications may differ from current materials. Treat the exact internal architecture as unconfirmed unless the company tells you directly.

- heyordo.com - product positioning, FAQ, privacy/thesis materials.
- LinkedIn - current company profile and public company/product descriptions.